Deploying a Multi-Service Node.js E-commerce Application

Using Terraform and Docker on AWS

Submitted by: Sagar Rathore · **Date:** 17 October 2026 · **Region:** ap-south-1 · **AWS account:** 400398151988
Docker Hub: saagarsaga · **Terraform:** v1.16.6 · **AWS provider:** ~> 5.60

1. Objective and outcome

Provision AWS infrastructure with Terraform and deploy five Dockerized Node.js services — four backends (user, products, orders, cart) and one frontend — onto an EC2 instance, with the frontend publicly accessible.

Outcome: **Completed.** All five images were built, pushed to Docker Hub and deployed by a single `terraform apply`. The frontend served "Frontend is Live" at `http://13.234.113.72` with all four backends reporting healthy, while backend ports 3001–3004 were verified unreachable from the internet.

Deliverable	Result
5 Dockerfiles, built and tested locally	**Done — all 6 endpoint checks passed**
Images tagged and pushed to Docker Hub	**Done — 5 public repos, linux/amd64 + linux/arm64**
VPC with a public subnet	**Done — vpc-0b28f142d2d1b77d3, 10.20.0.0/16**
EC2 instance hosting the containers	**Done — i-05f47627413ac11b3, t3.micro, Ubuntu 22.04**
Security groups (public 80/3000, internal 3001–3004)	**Done — sg-0fcc9192f27c43745, externally verified**
Docker installed and containers run via user-data	**Done — see the console log in section 6**
Frontend publicly accessible	**Done — HTTP 200, homepage renders**
Public IP / DNS printed as a Terraform output	**Done — 11 outputs, section 5**

2. Architecture



The frontend reaches each backend by container name over the private Docker bridge network, so no backend port needs to be exposed publicly. The security group permits 3001–3004 only from members of the group itself.

3. Application and containerisation

Each service is an Express app returning the sample response required by the brief, plus a `/health` endpoint and a demo data endpoint.

Service	Port	Sample response	Endpoints
user-service	3001	`user Service Running`	/, /health, /users
products-service	3002	`products Service Running`	/, /health, /products
orders-service	3003	`orders Service Running`	/, /health, /orders
cart-service	3004	`cart Service Running`	/, /health, /cart
frontend	3000 → 80	"Frontend is Live" page	/, /health, /api/status, /api/{service}

Dockerfile (user-service, representative of all five)

```
# ---- user-service ------------------------------------------------------------
# Small, production-only image: deps installed from the lockfile, app runs as
# the unprivileged "node" user that the base image already provides.
FROM node:20-alpine

ENV NODE_ENV=production \
    PORT=3001

WORKDIR /app

# Copy manifests first so the dependency layer is cached across code changes.
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY server.js ./

USER node

EXPOSE 3001

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget -q -O - http://127.0.0.1:3001/health > /dev/null 2>&1 || exit 1

CMD ["node", "server.js"]
```

Design choices: `node:20-alpine` for a small image (199 MB); manifests copied before source so the dependency layer caches; `npm ci` against a committed lockfile for reproducibility; the container runs as the unprivileged `node` user; and a `HEALTHCHECK` lets Docker Compose gate startup ordering.

Local build and test

`scripts/build-and-test.sh` builds all five images, starts them with Docker Compose and asserts every endpoint:

```
==> Endpoint checks
OK      user-service       http://localhost:3001/
OK      products-service   http://localhost:3002/
OK      orders-service     http://localhost:3003/
OK      cart-service       http://localhost:3004/
OK      frontend page      http://localhost:8080/
OK      aggregated state   http://localhost:8080/api/status
```

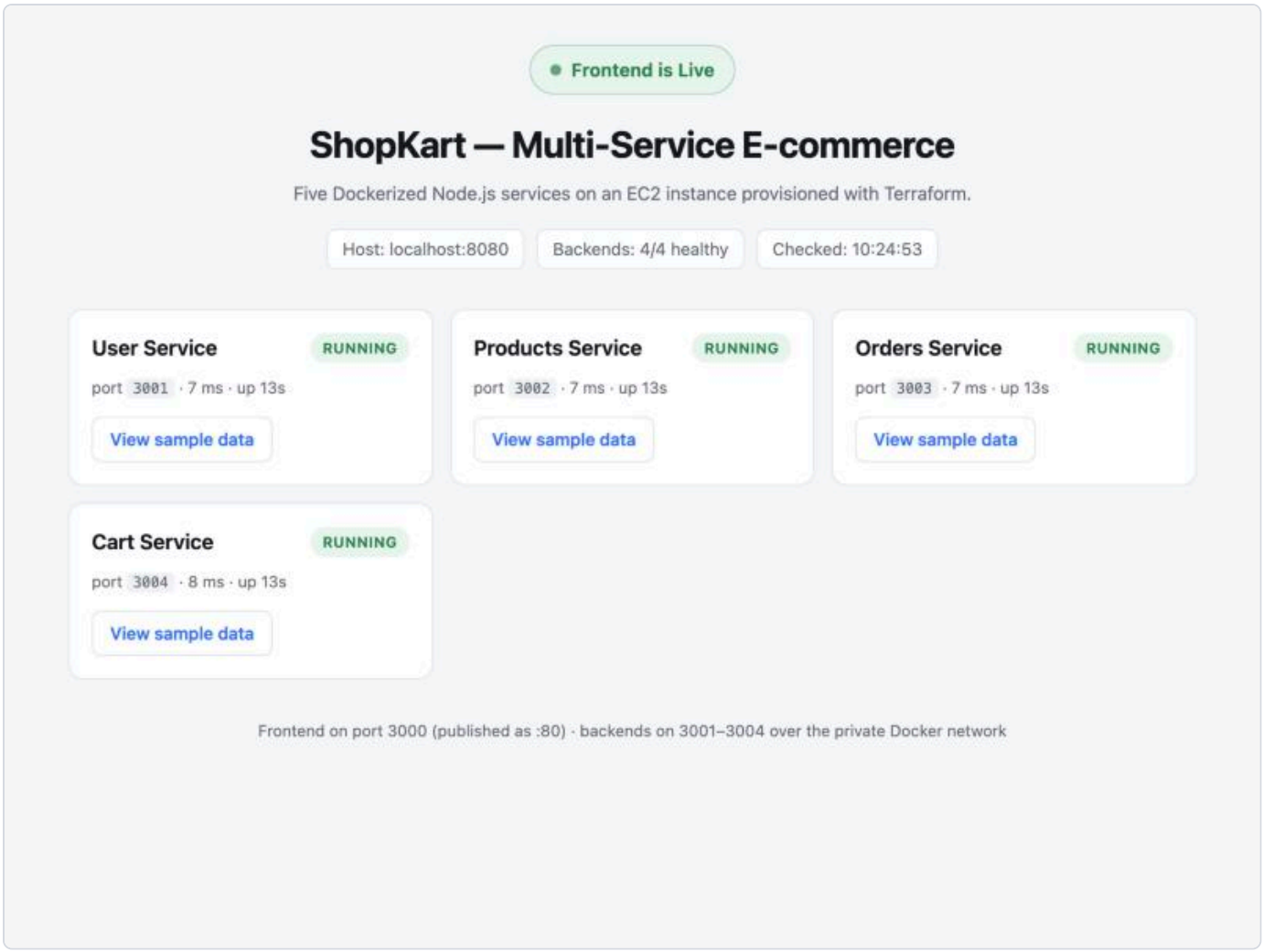


Figure 1 — The stack running locally under Docker Compose, 4/4 backends healthy.

Publishing to Docker Hub

Images were built with `docker buildx` for `linux/amd64` and `linux/arm64`. This matters: the build host is an Apple Silicon Mac, so a plain `docker build` would have produced arm64-only images that a t3.micro cannot run. The amd64 variant was verified locally before pushing (`uname -m` inside the container returned `x86_64` and the service answered correctly).

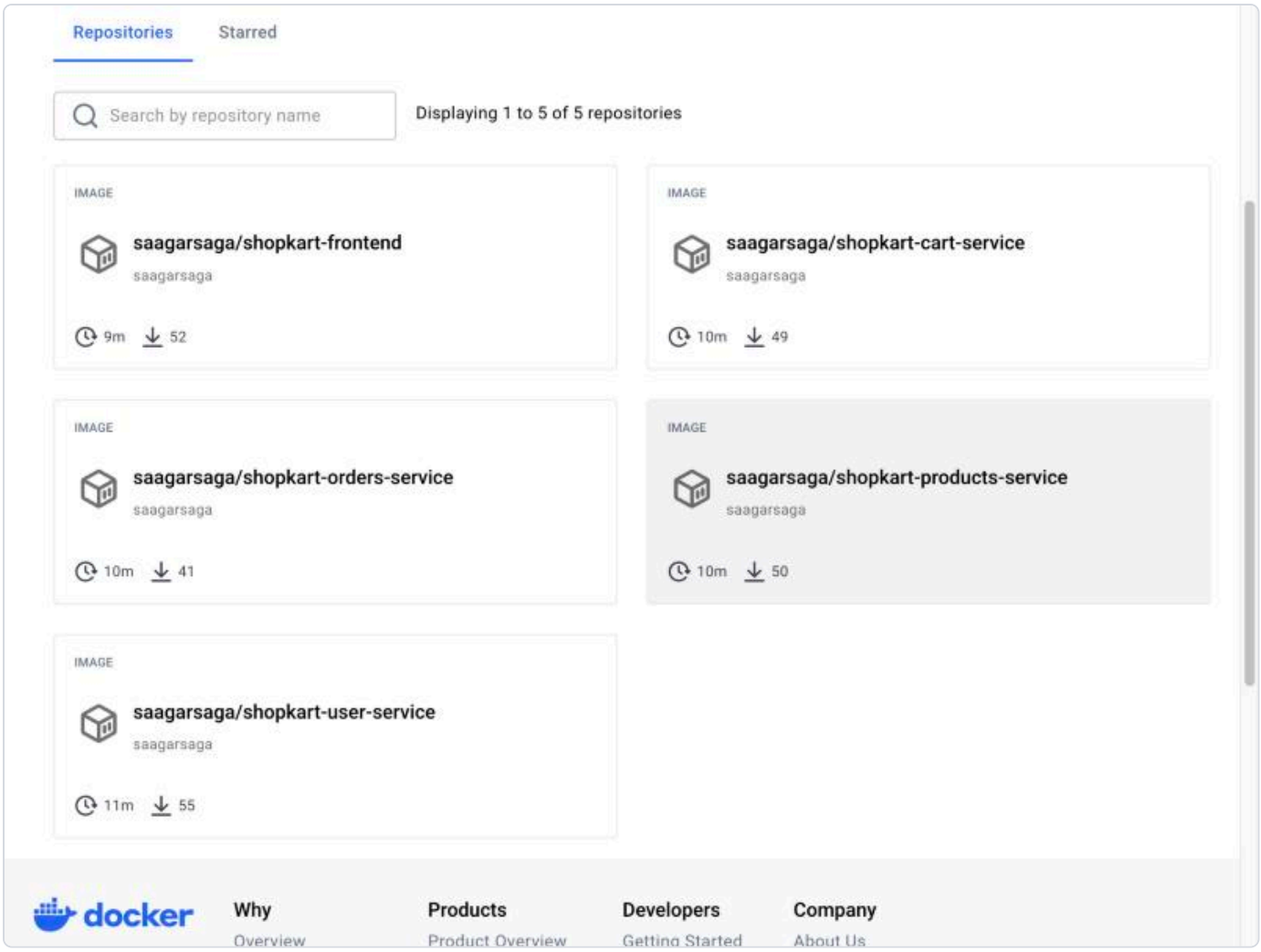


Figure 2 — The five public repositories on Docker Hub.

4. Infrastructure provisioning with Terraform

The configuration is split by concern: `network.tf`, `security.tf`, `ec2.tf`, `variables.tf`, `outputs.tf`, plus the `user_data.sh.tftpl` template. `terraform validate` passes and `terraform fmt -check` is clean.

Resources created (14)

Resource	Purpose
`aws_vpc.main`	VPC 10.20.0.0/16, DNS support and hostnames enabled
`aws_internet_gateway.main`	Internet access for the public subnet
`aws_subnet.public`	10.20.1.0/24 in ap-south-1a, auto-assigns public IPs
`aws_route_table.public` + association	Default route 0.0.0.0/0 to the internet gateway
`aws_security_group.app`	Frontend and inter-service access control
6 × `aws_vpc_security_group_ingress_rule`	80, 3000 public; 3001–3004 group-internal
`aws_vpc_security_group_egress_rule.all`	Outbound for apt and Docker Hub pulls
`aws_instance.app`	Ubuntu 22.04 t3.micro, encrypted gp3 root, IMDSv2 required

Security group design

The four backend rules use `referenced_security_group_id` pointing at the group itself rather than a CIDR, so 3001–3004 are reachable only from inside the group. This is what makes the "internal communication between services" requirement real rather than nominal — section 6 shows those ports refusing connections from the internet.

Direction	Port	Protocol	Source / destination	Description
Ingress	`80`	tcp	`203.0.113.4/32`	HTTP to the frontend container
Ingress	`3000`	tcp	`203.0.113.4/32`	Frontend on its native port 3000
Ingress	`3001`	tcp	`this security group`	Internal access to the user service
Ingress	`3002`	tcp	`this security group`	Internal access to the products service
Ingress	`3003`	tcp	`this security group`	Internal access to the orders service
Ingress	`3004`	tcp	`this security group`	Internal access to the cart service
Egress	`all`	all	`0.0.0.0/0`	Allow all outbound traffic

User-data: Docker install and container deployment

The rendered script performs seven steps: wait for cloud-init's apt locks; install Docker Engine from Docker's official apt repository (GPG key plus signed source list); enable the service; create the `shopkart-net` bridge network; pull all five images; run the four backends and the frontend with `--restart unless-stopped`; and finally run a smoke test against every `/health` endpoint, writing the result to `/home/ubuntu/deployment-status.txt`.

```
run_backend() {
  local name="$1" port="$2"
  log "Starting ${name} on port ${port}..."
  docker rm -f "${name}" >/dev/null 2>&1 || true
  docker run -d \
    --name "${name}" \
    --network "${NETWORK}" \
    --restart unless-stopped \
    -e "PORT=${port}" \
    -p "${port}:${port}" \
    "${DOCKERHUB_USER}/shopkart-${name}:${IMAGE_TAG}"
}

run_backend user-service 3001
run_backend products-service 3002
run_backend orders-service 3003
run_backend cart-service 3004

# --- 6. Run the frontend ------------------------------------------------
# Published on both the configured host port (80 by default) and 3000, and
# pointed at the backends by their container names on the Docker network.
log "Starting frontend..."
docker rm -f frontend >/dev/null 2>&1 || true

# Always publish 3000; add the configured host port too when it differs.
PORT_ARGS=(-p "3000:3000")
if [ "${FRONTEND_HOST_PORT}" != "3000" ]; then
  PORT_ARGS+=(-p "${FRONTEND_HOST_PORT}:3000")
fi

docker run -d \
  --name frontend \
  --network "${NETWORK}" \
  --restart unless-stopped \
  -e "PORT=3000" \
  -e "USER_SERVICE_URL=http://user-service:3001" \
  -e "PRODUCTS_SERVICE_URL=http://products-service:3002" \
  -e "ORDERS_SERVICE_URL=http://orders-service:3003" \
  -e "CART_SERVICE_URL=http://cart-service:3004" \
  "${PORT_ARGS[@]}" \
  "${DOCKERHUB_USER}/shopkart-frontend:${IMAGE_TAG}"
```

Excerpt from `terraform/user_data.sh.tftpl`, shown as rendered (Terraform's `$$` escapes resolved).

5. Deployment

```
$ terraform init && terraform plan -out=tfplan && terraform apply tfplan

aws_instance.app: Creation complete after 13s [id=i-05f47627413ac11b3]

Apply complete! Resources: 14 added, 0 changed, 0 destroyed.
```

Terraform outputs

```
frontend_url = "http://13.234.113.72"
frontend_url_port_3000 = "http://13.234.113.72:3000"
health_endpoints = {
  "all_services" = "http://13.234.113.72/api/status"
  "frontend_health" = "http://13.234.113.72/health"
}
instance_id = "i-05f47627413ac11b3"
instance_public_dns = "ec2-13-234-113-72.ap-south-1.compute.amazonaws.com"
instance_public_ip = "13.234.113.72"
public_subnet_id = "subnet-0e124d1e8ee18e061"
security_group_id = "sg-0fcc9192f27c43745"
ssh_command = "No key pair configured; set key_pair_name and allowed_ssh_cidr to enable SSH."
verify_backends_command = "ssh ubuntu@13.234.113.72 'cat deployment-status.txt && docker ps'"
vpc_id = "vpc-0b28f142d2d1b77d3"
```

Provisioned resources, confirmed via the AWS API

```
-------------------------------------------------------------------------
|                            DescribeVpcs                               |
+--------------+---------------+------------+--------------------------+
|     Cidr     |     Name      |   State    |          VpcId           |
+--------------+---------------+------------+--------------------------+
|  10.20.0.0/16|  shopkart-vpc |  available |  vpc-0b28f142d2d1b77d3   |
+--------------+---------------+------------+--------------------------+
```

```
-----------------------------------------------------------------------------
|                           DescribeSubnets                                 |
+-------------+---------------+-----------+-----------------------------+
|     AZ      |     Cidr      | PublicIP  |          SubnetId           |
+-------------+---------------+-----------+-----------------------------+
|  ap-south-1a|  10.20.1.0/24 |  True     |  subnet-0e124d1e8ee18e061   |
+-------------+---------------+-----------+-----------------------------+
```

```
-----------------------------------------
|           DescribeInstances           |
+------------+--------------------------+
|  AMI       |  ami-0644c826df601fd4f   |
|  AZ        |  ap-south-1a             |
|  Id        |  i-05f47627413ac11b3     |
|  PrivateIP |  10.20.1.86              |
|  PublicIP  |  13.234.113.72           |
|  State     |  running                 |
|  Type      |  t3.micro                |
+------------+--------------------------+
```

6. Verification

6.1 Public frontend access

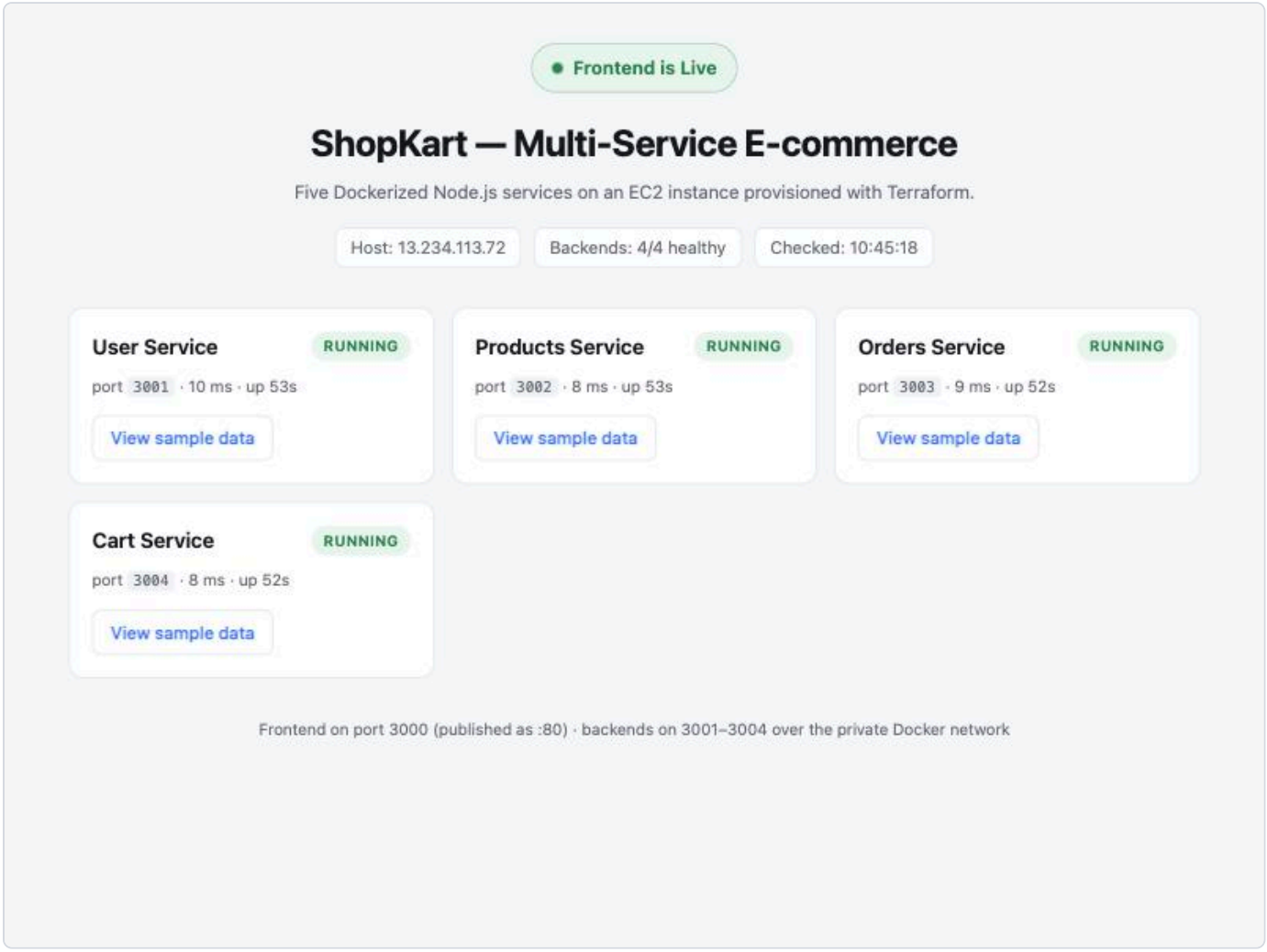


Figure 3 — The frontend served from the EC2 public IP, reporting 4/4 backends healthy.

6.2 Backend services responding through the frontend

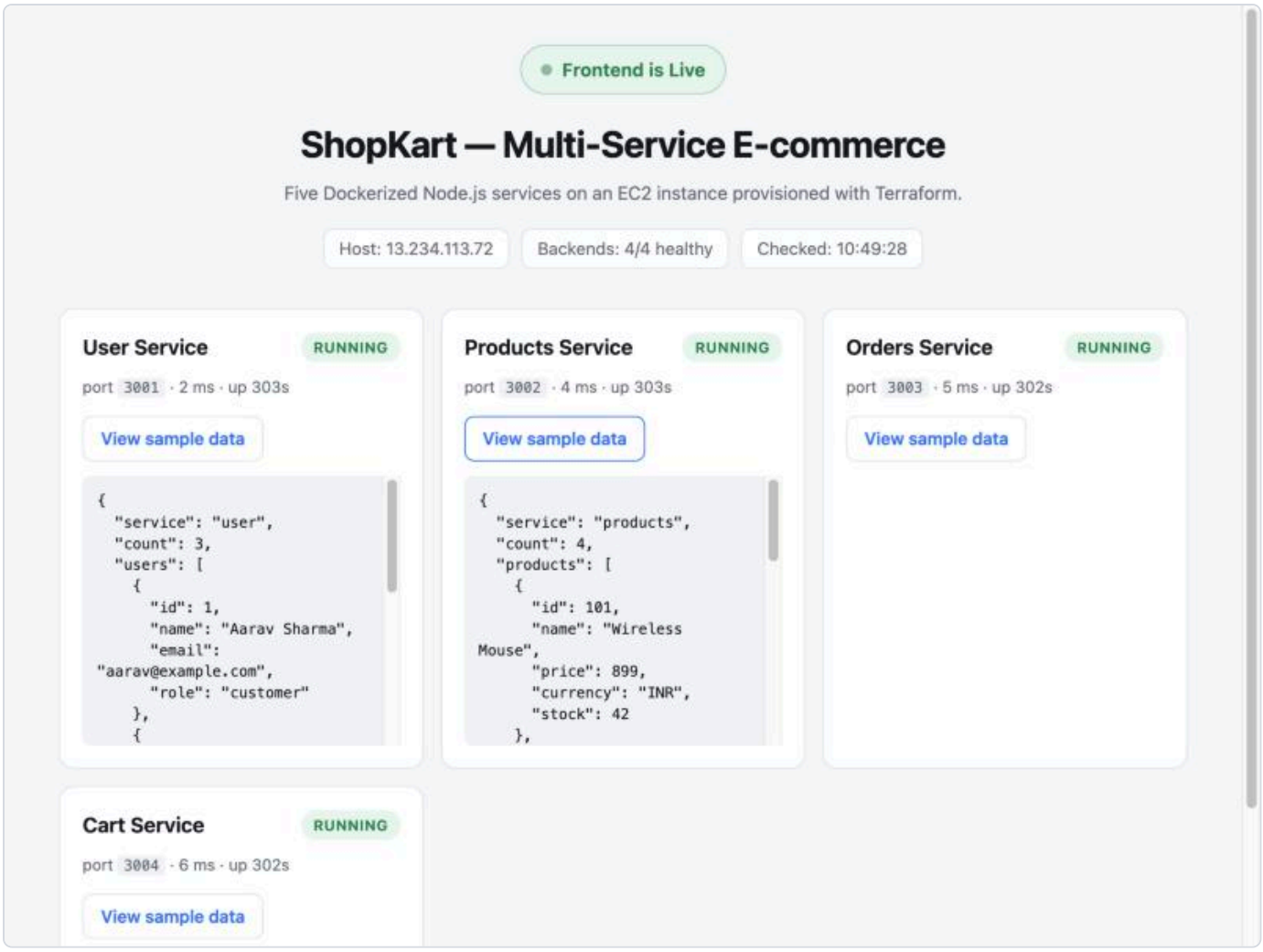


Figure 4 — Live JSON from the user and products services, proxied over the private Docker network.

6.3 Endpoint checks

```
GET http://13.234.113.72/              -> "Frontend is Live" homepage
GET http://13.234.113.72/api/status    -> 4/4 backends healthy
GET http://13.234.113.72:3000/health   -> HTTP 200
GET http://13.234.113.72/api/user      -> 3 users
GET http://13.234.113.72/api/products  -> 4 products
GET http://13.234.113.72/api/orders    -> 3 orders
GET http://13.234.113.72/api/cart      -> 2 carts

Backend ports probed from the public internet (must be refused):
  :3001 blocked    :3002 blocked    :3003 blocked    :3004 blocked
```

The last block is the security-group evidence: the backends are running and answering the frontend, yet their ports are not reachable from outside the group.

6.4 Containers running on the instance

Captured from the EC2 serial console, which records the user-data run. No SSH key was configured, so this is the deployment's own log rather than an interactive session:

```
ShopKart deployment status - 2026-10-17T05:14:27+00:00
OK      user-service      (http://localhost:3001/health)
OK      products-service  (http://localhost:3002/health)
OK      orders-service    (http://localhost:3003/health)
OK      cart-service      (http://localhost:3004/health)
OK      frontend          (http://localhost:80/health)
NAMES              IMAGE                                          STATUS
PORTS
frontend           saagarsaga/shopkart-frontend:1.0.0           Up Less than a second (health: starting)
0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp, 0.0.0.0:80->3000/tcp, [::]:80->3000/tcp
cart-service       saagarsaga/shopkart-cart-service:1.0.0       Up Less than a second (health: starting)
0.0.0.0:3004->3004/tcp, [::]:3004->3004/tcp
orders-service     saagarsaga/shopkart-orders-service:1.0.0     Up 1 second (health: starting)
0.0.0.0:3003->3003/tcp, [::]:3003->3003/tcp
products-service   saagarsaga/shopkart-products-service:1.0.0   Up 1 second (health: starting)
0.0.0.0:3002->3002/tcp, [::]:3002->3002/tcp
user-service       saagarsaga/shopkart-user-service:1.0.0       Up 2 seconds (health: starting)
0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp
[2026-10-17T05:14:27+00:00] === ShopKart deployment finished ===
```

6.5 Docker installation and image pulls

```
[2026-10-17T05:14:01+00:00] Docker installed: Docker version 29.9.1, build dfc520d
[2026-10-17T05:14:01+00:00] Docker network ready: shopkart-net
[2026-10-17T05:14:01+00:00] Pulling saagarsaga/shopkart-user-service:1.0.0...
Status: Downloaded newer image for saagarsaga/shopkart-user-service:1.0.0
[2026-10-17T05:14:07+00:00] Pulling saagarsaga/shopkart-products-service:1.0.0...
Status: Downloaded newer image for saagarsaga/shopkart-products-service:1.0.0
[2026-10-17T05:14:11+00:00] Pulling saagarsaga/shopkart-orders-service:1.0.0...
Status: Downloaded newer image for saagarsaga/shopkart-orders-service:1.0.0
[2026-10-17T05:14:15+00:00] Pulling saagarsaga/shopkart-cart-service:1.0.0...
Status: Downloaded newer image for saagarsaga/shopkart-cart-service:1.0.0
[2026-10-17T05:14:19+00:00] Pulling saagarsaga/shopkart-frontend:1.0.0...
Status: Downloaded newer image for saagarsaga/shopkart-frontend:1.0.0
```

7. Security notes

- Backend ports 3001–3004 accept traffic only from the security group itself; verified externally as refused.
- No SSH ingress rule is created unless `allowed_ssh_cidr` is set explicitly; this deployment ran with no SSH exposure at all.
- IMDSv2 is required on the instance and the root EBS volume is encrypted.
- Containers run as a non-root user with `--restart unless-stopped`.
- `allowed_http_cidr` was narrowed from `0.0.0.0/0` to a single /32 after the public-access evidence above was captured, so the frontend was not left open to the internet.
- The Docker Hub token path is optional and unused here: the images are public, which keeps credentials out of user-data (readable through the metadata service).

8. Teardown

Every resource lives inside the dedicated VPC, so a single command removes all 14 and leaves nothing behind:

```
$ cd terraform && terraform destroy
```

9. Mapping to the evaluation criteria

Criterion	Marks	Evidence
VPC, subnet and security group setup	**10**	`network.tf`, `security.tf`; sections 4 and 5
EC2 provisioning with Docker setup	**10**	`ec2.tf` and user-data steps 1–2; section 6.5
Container deployment via user-data	**10**	User-data steps 3–7; section 6.4
Public access to the frontend	**10**	Figures 3 and 4; section 6.3
Documentation	**10**	This report, the repository README and inline comments throughout

Appendix — repository layout

```
.
|-- README.md                    full setup, deploy and troubleshooting guide
|-- docker-compose.yml           local build-and-test harness
|-- docs/                        this report and all captured evidence
|-- scripts/
|   |-- build-and-test.sh        build 5 images, run them, assert endpoints
|   `-- push-images.sh           multi-arch buildx build and push
|-- services/
|   |-- user-service/            server.js, package.json, Dockerfile
|   |-- products-service/
|   |-- orders-service/
|   |-- cart-service/
|   `-- frontend/                + public/index.html
`-- terraform/
    |-- versions.tf  provider.tf  variables.tf
    |-- network.tf   security.tf  ec2.tf
    |-- user_data.sh.tftpl       Docker install + container deployment
    |-- outputs.tf               public IP / DNS / URLs
    `-- terraform.tfvars.example
```